subtractive mixture models

representation, learning & inference

antonio vergari (he/him)

@nolovedeeplearning

8th June 2026 - **Trento**

thanks to...

Lorenzo Loconte
U of Edinburgh

Lena Zellinger
U of Edinburgh

Aleksanteri Sladek
Aalto U

Gennaro Gala
TU Eindhoven

Adrian Javaloy
U of Edinburgh

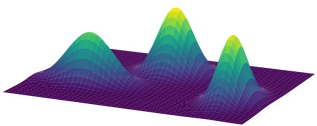
and moar...

april

april-tools.github.io

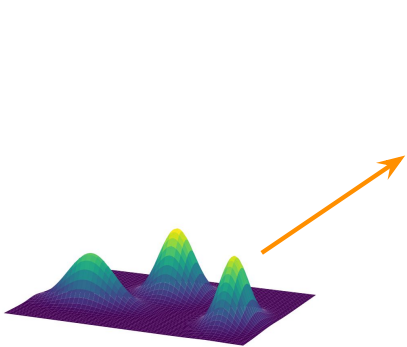
today's topic...

swiss-army knife of prob ML



(hierarchical)
mixtures

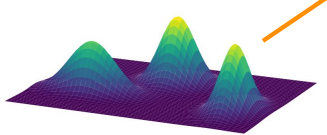
generalizing them as computational graphs



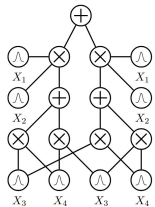
(deep) circuits

(hierarchical) mixtures

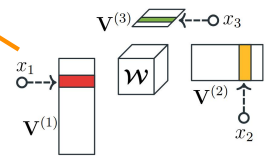
a single formalism for many models



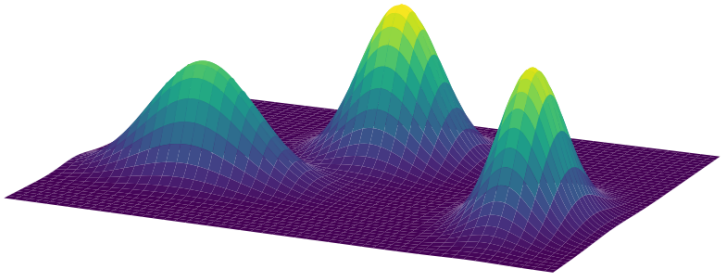
(hierarchical)
mixtures



(deep)
circuits

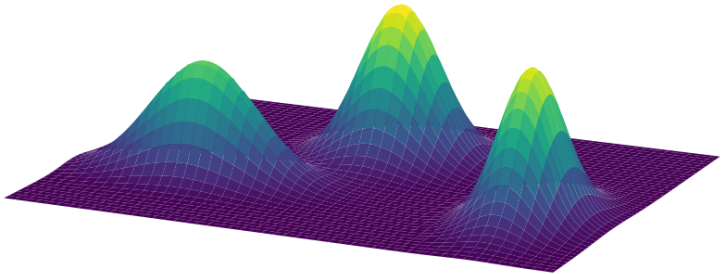


(hierarchical)
tensor factorizations



who knows mixture models?

image taken from Hao Tang's course on ASR



who loves mixture models?

image taken from Hao Tang's course on ASR

Hierarchical Gaussian Mixture Model Splatting for Efficient and Part Controllable 3D Generation

Qitong Yang, Mingtao Feng, Zijie Wu, Weisheng Dong, Fangfang Wu, Yaonan Wang, Ajmal Mian;
Proceedings of the Computer Vision and Pattern Recognition Conference (CVPR), 2025, pp. 11104-11114

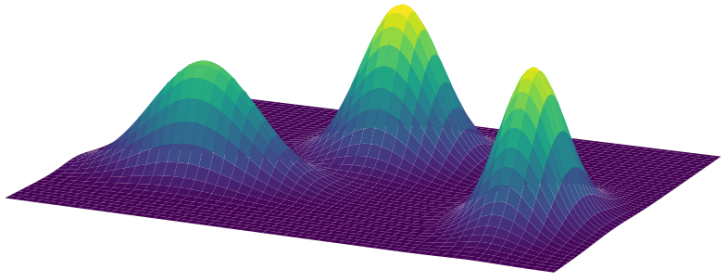
Inversion of nitrogen and phosphorus contents in cotton leaves based on the Gaussian mixture model and differences in hyperspectral features of UAV

Lei Peng ✉, Hui-Nan Xin ✉, Cai-Xia Lv ✉, Na Li ✉, Yong-Fu Li ✉, Qing-Long Geng ✉, Shu-Huang Chen ✉, Ning Lai ✉

Gaussian Mixture Flow Matching Models

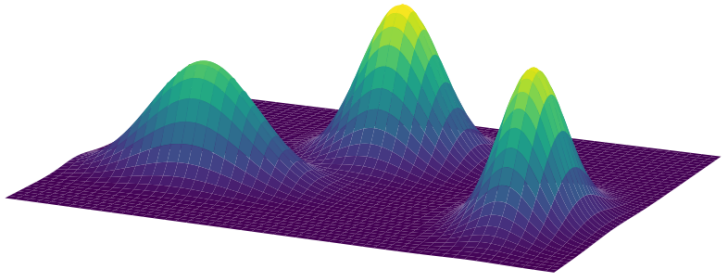
Hansheng Chen [1] **Kai Zhang** [2] **Hao Tan** [2] **Zexiang Xu** [3] **Fujun Luan** [2]
Leonidas Guibas [1] **Gordon Wetzstein** [1] **Sai Bi** [2]

mixture models are everywhere
(still in 2025)



$$c(\mathbf{X}) = \sum_{i=1}^{K} w_i c_i(\mathbf{X}), \quad \text{with} \quad w_i \geq 0, \quad \sum_{i=1}^{K} w_i = 1$$

image taken from Hao Tang's course on ASR



$$c(\mathbf{X}) = \sum_{i=1}^{K} w_i c_i(\mathbf{X}), \quad \text{with} \quad w_i \geq 0, \quad \sum_{i=1}^{K} w_i = 1$$

image taken from Hao Tang's course on ASR

$$\int \sum_i w_i p_i(\mathbf{x}) d\mathbf{x} = \sum_i w_i \int p_i(\mathbf{x}) d\mathbf{x}$$

mixture models can enable tractable inference

(if components are tractable, e.g., for marginals)

Hierarchical Decompositional Mixtures of Variational Autoencoders

Ping Liang Tan [1 2] Robert Peharz [1]

Mixtures of Laplace Approximations for Improved *Post-Hoc* Uncertainty in Deep Learning

Runa Eschenhagen[*1] Erik Daxberger[*,m] Philipp Hennig[1,m] Agustinus Kristiadi[*]

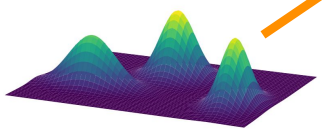
Efficient Mixture Learning in Black-Box Variational Inference

Alexandra Hotti [* 1 2 3] Oskar Kviman [* 1 2] Ricky Molén [1 2] Víctor Elvira [4] Jens Lagergren [1 2]

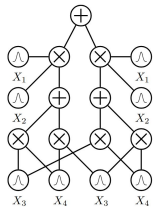
mixture models can enable tractable inference

(even in larger approximate inference pipelines)

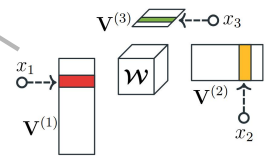
compile mixtures into circuits...



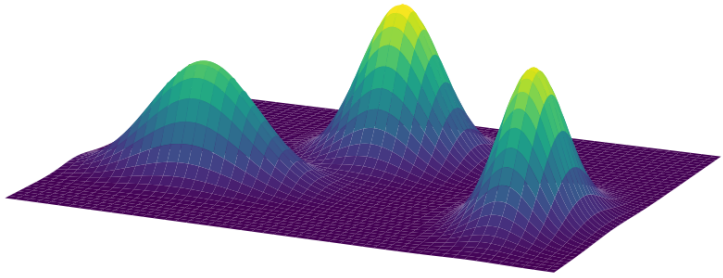
(hierarchical)
mixtures



(deep)
circuits



(hierarchical)
tensor factorizations

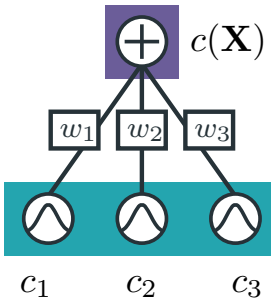


$$c(\mathbf{X}) = \sum_{i=1}^{K} w_i c_i(\mathbf{X}), \quad \text{with} \quad w_i \geq 0, \quad \sum_{i=1}^{K} w_i = 1$$

image taken from Hao Tang's course on ASR

additive MMs

are so cool!



easily represented as shallow PCs

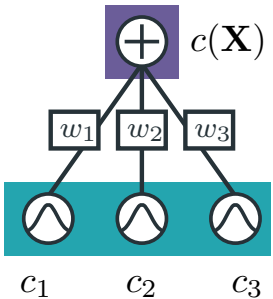
these are ***monotonic*** PCs

if marginals/conditionals are tractable for the components, they are tractable for the MM

they are ***universal approximators***...

additive MMs

are so cool!



easily represented as shallow PCs

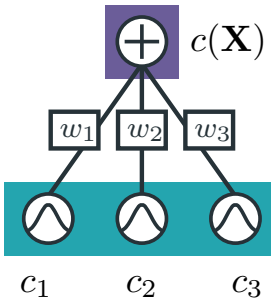
these are ***monotonic*** PCs

if marginals/conditionals are tractable for the components, they are tractable for the MM

they are ***universal approximators***...

additive MMs

are so cool!



easily represented as shallow PCs

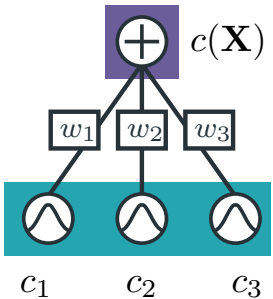
these are ***monotonic*** PCs

if marginals/conditionals are tractable for the components, they are tractable for the MM

they are ***universal approximators***...

additive MMs

are so cool!



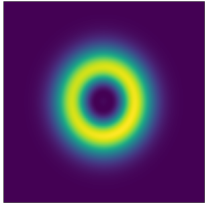
easily represented as shallow PCs

these are ***monotonic*** PCs

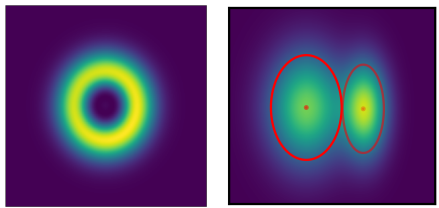
if marginals/conditionals are tractable for the components, they are tractable for the MM

they are ***universal approximators***...

however...

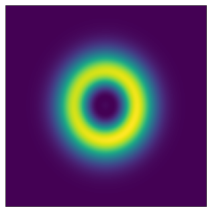


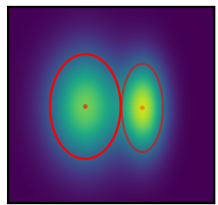
however...



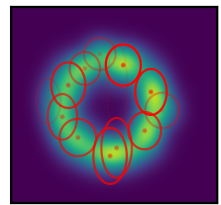
GMM ($K = 2$)

however...



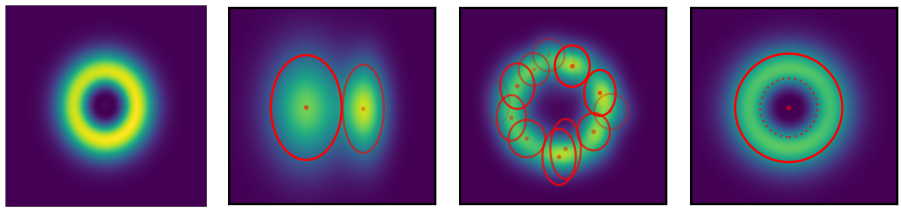


GMM ($K = 2$)



GMM ($K = 16$)

however...



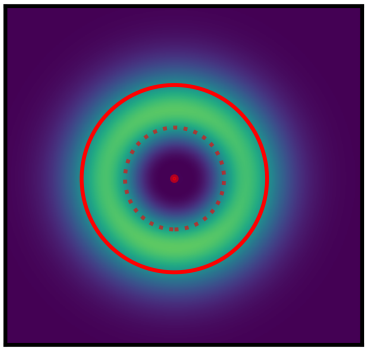
GMM ($K = 2$) GMM ($K = 16$) nGMM^2 ($K = 2$)

spoiler

shallow mixtures
with negative parameters
can be *exponentially more compact* than
deep ones with positive parameters

Loconte et al., "Subtractive Mixture Models via Squaring: Representation and Learning", ICLR, 2024

subtractive MMs



also called negative/signed/**subtractive** MMs

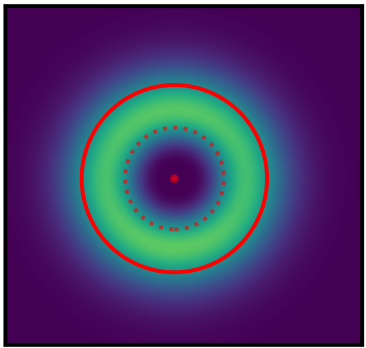
⇒ *or* ***non-monotonic*** *circuits,...*

issue: how to preserve non-negative outputs?

well understood for simple parametric forms
e.g., Weibulls, Gaussians

⇒ *constraints on variance, mean*

subtractive MMs



also called negative/signed/**subtractive** MMs

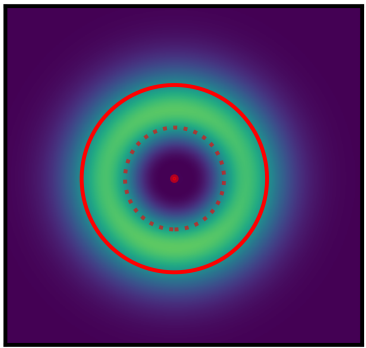
⇒ *or* ***non-monotonic*** *circuits,...*

issue: how to preserve non-negative outputs?

well understood for simple parametric forms
e.g., Weibulls, Gaussians

⇒ *constraints on variance, mean*

subtractive MMs



also called negative/signed/**subtractive** MMs

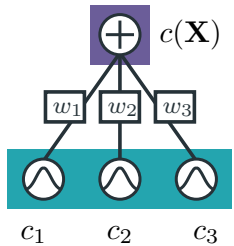
$\Rightarrow$ *or* ***non-monotonic*** *circuits,...*

issue: how to preserve non-negative outputs?

well understood for simple parametric forms
e.g., Weibulls, Gaussians

$\Rightarrow$ *constraints on variance, mean*

subtractive MMs as circuits

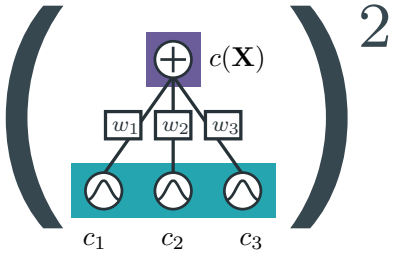


a **non-monotonic** smooth and (structured) decomposable circuit

$\Rightarrow$ *possibly with negative outputs*

$$c(\mathbf{X}) = \sum_{i=1}^{K} w_i c_i(\mathbf{X}), \qquad w_i \in \mathbb{R},$$

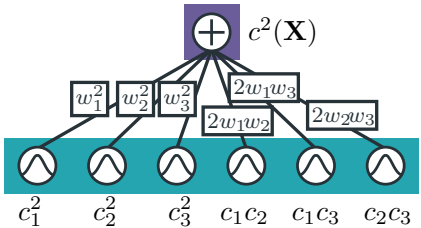
squaring shallow MMs



$$c^2(\mathbf{X}) = \left(\sum_{i=1}^{K} w_i c_i(\mathbf{X})\right)^2$$

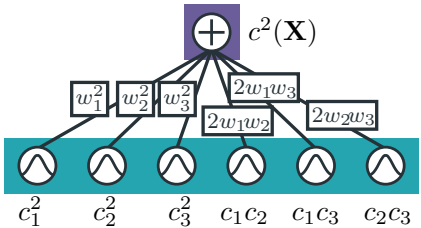
$\Rightarrow$ *ensure non-negative output*

squaring shallow MMs



$$c^2(\mathbf{X}) = \left(\sum_{i=1}^{K} w_i c_i(\mathbf{X})\right)^2$$
$$= \sum_{i=1}^{K}\sum_{j=1}^{K} w_i w_j c_i(\mathbf{X}) c_j(\mathbf{X})$$

squaring shallow MMs

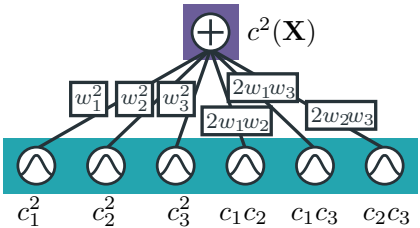


$$c^2(\mathbf{X}) = \left(\sum_{i=1}^{K} w_i c_i(\mathbf{X})\right)^2 = \sum_{i=1}^{K}\sum_{j=1}^{K} w_i w_j c_i(\mathbf{X}) c_j(\mathbf{X})$$

still a smooth and (str) decomposable PC with $\mathcal{O}(K^2)$ components!

$\Rightarrow$ *but still $\mathcal{O}(K)$ parameters*

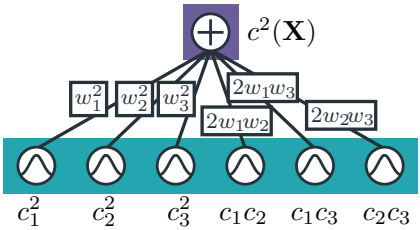
squaring shallow MMs



$$c^2(\mathbf{X}) = \left(\sum_{i=1}^{K} w_i c_i(\mathbf{X})\right)^2$$
$$= \sum_{i=1}^{K}\sum_{j=1}^{K} w_i w_j c_i(\mathbf{X}) c_j(\mathbf{X})$$

how to ***renormalize***?

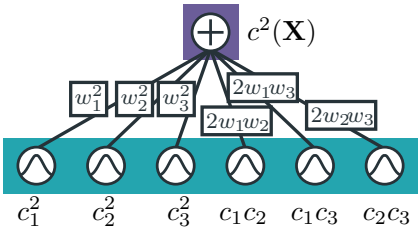
squaring shallow MMs



$$c^2(\mathbf{X}) = \left(\sum_{i=1}^{K} w_i c_i(\mathbf{X})\right)^2 = \sum_{i=1}^{K}\sum_{j=1}^{K} w_i w_j c_i(\mathbf{X}) c_j(\mathbf{X})$$

to ***renormalize***, we have to compute $\sum_i \sum_j w_i w_j \int c_i(\mathbf{x}) c_j(\mathbf{x}) d\mathbf{x}$

squaring shallow MMs



$$c^2(\mathbf{X}) = \left(\sum_{i=1}^{K} w_i c_i(\mathbf{X})\right)^2 = \sum_{i=1}^{K}\sum_{j=1}^{K} w_i w_j c_i(\mathbf{X}) c_j(\mathbf{X})$$

to ***renormalize***, we have to compute $\sum_i \sum_j w_i w_j \int c_i(\mathbf{x}) c_j(\mathbf{x}) d\mathbf{x}$

$\Longrightarrow$ *closed-form for e.g., if c_i, c_j are **exponential families...!***

wait...!

how do we learn them?

wait...!

how do we learn them?

$\Rightarrow$ ***by maximizing the (log-)likelihood***

which parameters?

how to reparameterize non-monotonic mixtures/circuits

Input functions.

Sum unit parameters.

which parameters?

how to reparameterize non-monotonic mixtures/circuits

Input functions. Each input can be a different parametric ***function***

$\Rightarrow$ *Bernoullis, Categoricals, Gaussians,* ***polynomials****, small NNs, ...*

Sum unit parameters.

which parameters?

how to reparameterize non-monotonic mixtures/circuits

Input functions. Each input can be a different parametric ***function***

Sum unit parameters. They can be negative, i.e., $w_i \in \mathbb{R}$ and we we need to renormalize the ***negative log likelihood*** loss after squaring

$$\min_{\theta} - \left(\sum_{i=1}^{N} 2 \log c_\theta(\mathbf{x}^{(i)}) - \log \int c_\theta^2(\mathbf{x}^{(i)}) \, d\mathbf{X} \right)$$

wait...!

how do we learn them?

$\Rightarrow$ ***by maximizing the (log-)likelihood***

wait...!

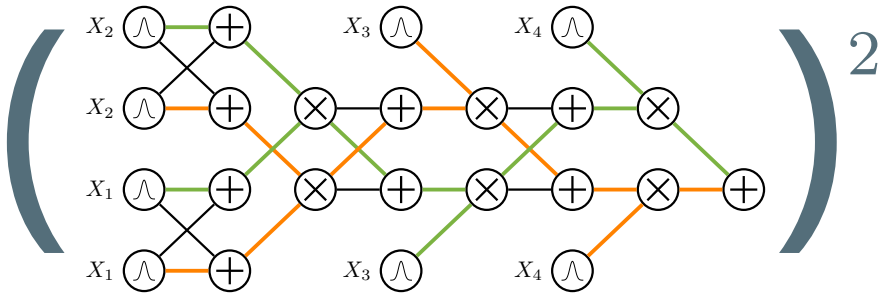
how do we learn them?

⇒ ***by maximizing the (log-)likelihood***

just SGD your way as usual!

⇒ ***or any other gradient-based optimizer***

what about deep mixtures/circuits?



how to efficiently square (and *renormalize*) a deep PC?

Loconte et al., "Subtractive Mixture Models via Squaring: Representation and Learning", ICLR, 2024

compositional inference I


```
1  from cirkit.symbolic.functional import integrate, multiply
2
3  #
4  # create a deep circuit
5  c = build_symbolic_circuit('quad-tree-4')
6
7  #
8  # compute the partition function of c^2
9  def renormalize(c):
10     c2 = multiply(c, c)
11     return integrate(c2)
```

structural properties

smoothness

decomposability

property C

property D

Vergari *et al., "A Compositional Atlas of Tractable Circuit Operations for Probabilistic Inference", NeurIPS, 2021*

structural properties

smoothness

decomposability

compatibility

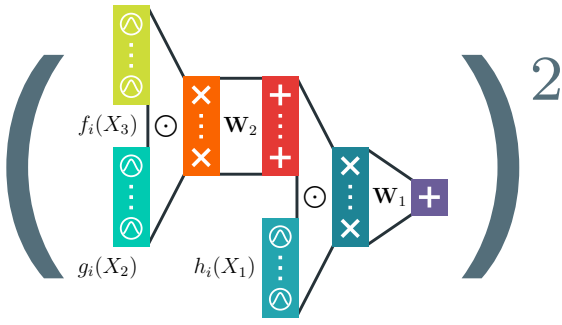
property D

Integrals involving two or more functions:
e.g., expectations

$$\mathbb{E}_{\mathbf{x}\sim p} f(\mathbf{x}) = \int p(\mathbf{x}) \; f(\mathbf{x}) \; \mathrm{d}\mathbf{x}$$

when both $p(\mathbf{x})$ and $f(\mathbf{x})$ are circuits

***Vergari** et al., "A Compositional Atlas of Tractable Circuit Operations for Probabilistic Inference", NeurIPS, 2021*

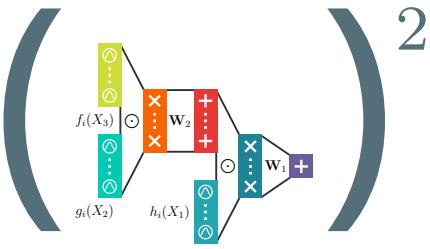


how to efficiently square (and *renormalize*) a deep PC?

Loconte et al., "Subtractive Mixture Models via Squaring: Representation and Learning", ICLR, 2024

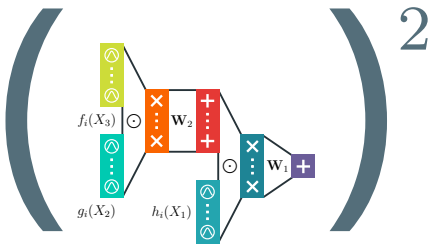
squaring deep PCs

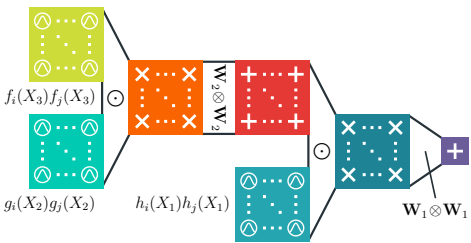
the tensorized way



squaring deep PCs

the tensorized way

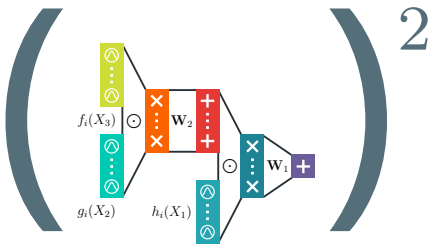


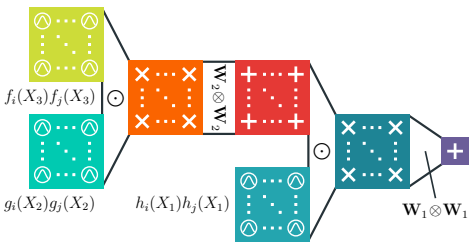


squaring a circuit = squaring layers

squaring deep PCs

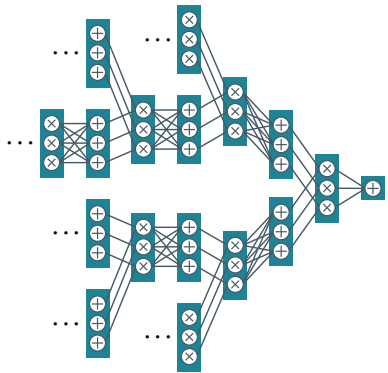
the tensorized way





exactly compute $\int \boldsymbol{c}(\mathrm{x})\boldsymbol{c}(\mathrm{x})d\mathrm{X}$ in time $O(\boldsymbol{L}\boldsymbol{K}^2)$

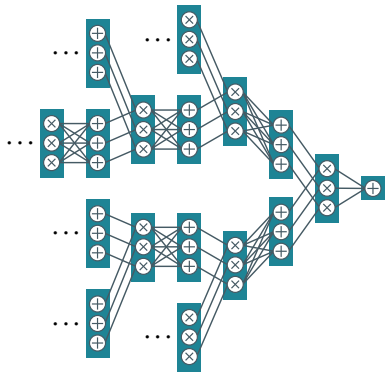
theorem I

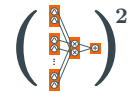


$\exists\, p'$ **requiring exponentially large monotonic circuits...**

Loconte et al., "Subtractive Mixture Models via Squaring: Representation and Learning", ICLR, 2024

theorem I



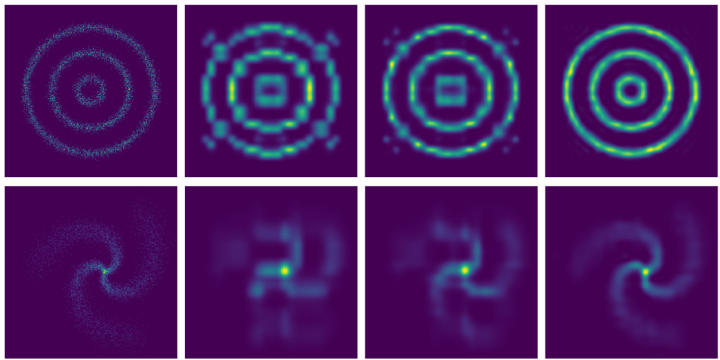


...but compact

squared non-monotonic circuits

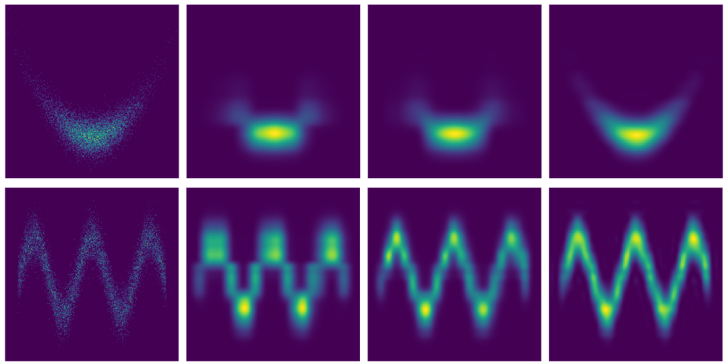
Loconte et al., "Subtractive Mixture Models via Squaring: Representation and Learning", ICLR, 2024

more expressive?



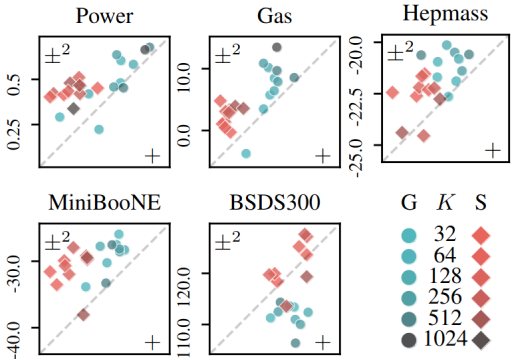
data | monoPC | monoPC^2 | non $-$ monoPC^2

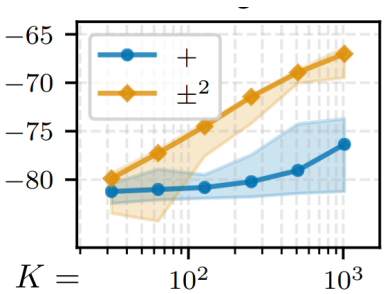
more expressive?



how more expressive?

real-world data

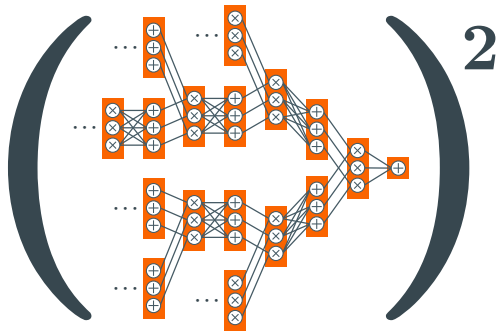




theorem II

$\exists\, p''$ **requiring exponentially large**

squared non-mono circuits...

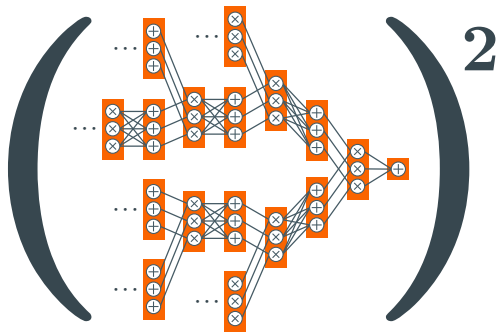


Loconte, Mengel, and Vergari, "Sum of Squares Circuits", AAAI, 2025

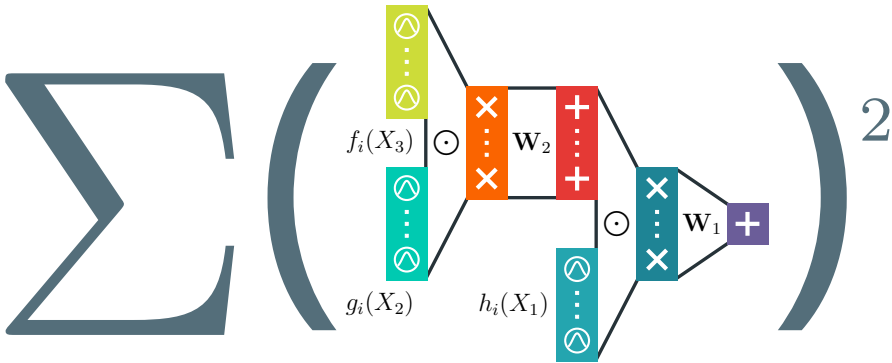
theorem II

...but compact

monotonic circuits...!

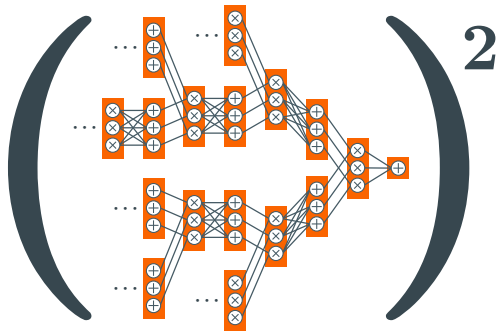


Loconte, Mengel, and Vergari, "Sum of Squares Circuits", AAAI, 2025



what if we use more that one square?

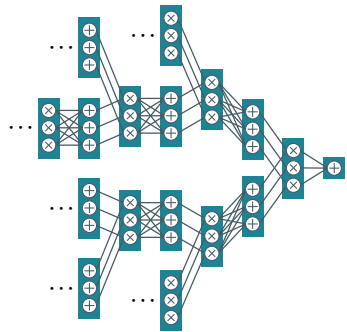
theorem III

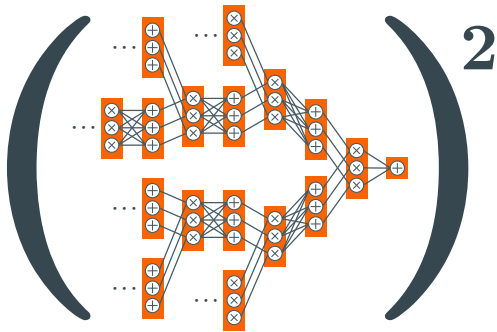


$\exists\, p'''$ **requiring exponentially large** **squared non-mono circuits...**

Loconte, Mengel, and Vergari, "Sum of Squares Circuits", AAAI, 2025

theorem III

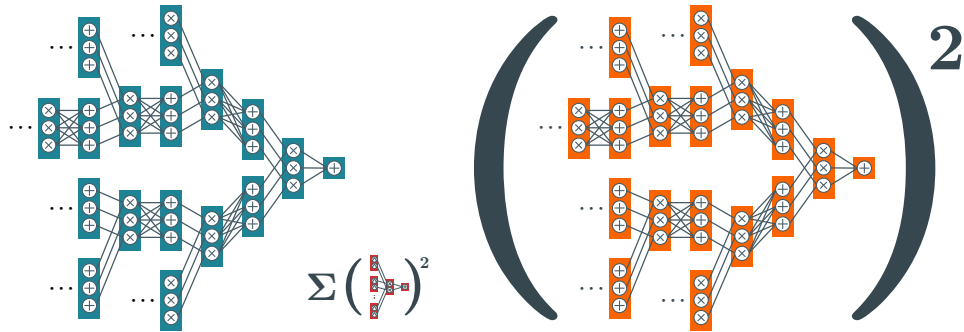




…exponentially large monotonic circuits…

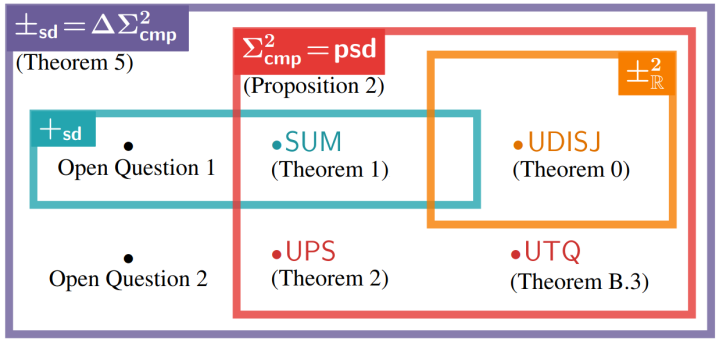
Loconte, Mengel, and Vergari, "Sum of Squares Circuits", AAAI, 2025

theorem III



...but compact SOS circuits...!

Loconte, Mengel, and Vergari, "Sum of Squares Circuits", AAAI, 2025



a hierarchy of subtractive mixtures

Loconte, Mengel, and Vergari, "Sum of Squares Circuits", AAAI, 2025

we can define circuits (and hence mixtures) over the Complex:

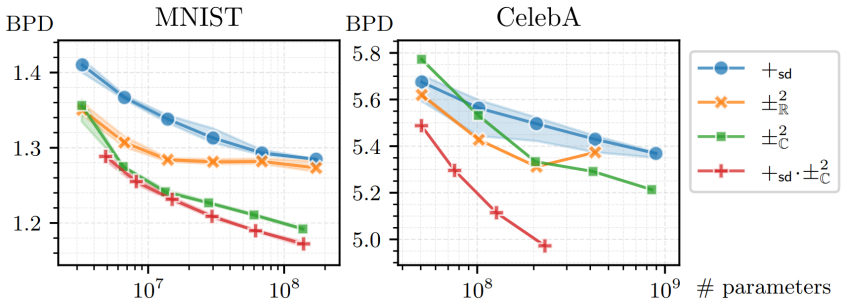
$$c^2(\mathbf{x}) = c(\mathbf{x})^\dagger c(\mathbf{x}), \qquad c(\mathbf{x}) \in \mathbb{C}$$

and then we can note that they can be written as a SOS form

$$c^2(\mathbf{x}) = r(\mathbf{x})^2 + i(\mathbf{x})^2, \qquad r(\mathbf{x}), i(\mathbf{x}) \in \mathbb{R}$$

complex circuits are SOS (and scale better!)

Loconte, Mengel, and Vergari, "Sum of Squares Circuits", AAAI, 2025



complex circuits are SOS (and scale better!)

Loconte, Mengel, and Vergari, "Sum of Squares Circuits", AAAI, 2025

takeaway

“use squared mixtures over complex numbers (and you get a SOS for free)”

takeaway

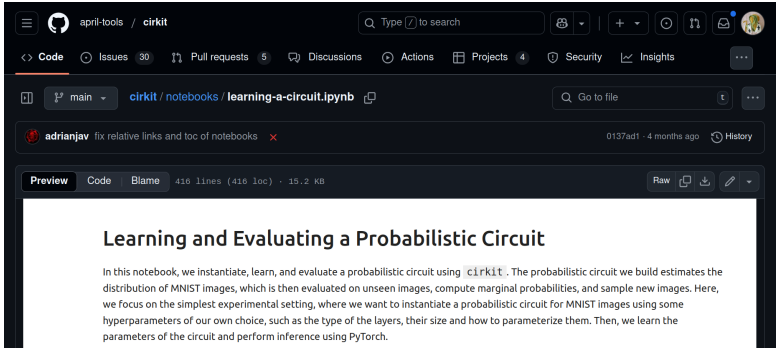
"use squared mixtures over complex numbers (and you get a SOS for free)"

$\Rightarrow$ ***but how to implement them?***

compositional inference I

cirkit

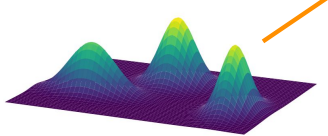
```
from cirkit.symbolic.functional import integrate, multiply,
    conjugate

# create a deep circuit with complex parameters
c = build_symbolic_complex_circuit('quad-tree-4')

# compute the partition function of c^2
def renormalize(c):
    c1 = conjugate(c)
    c2 = multiply(c, c1)
    return integrate(c2)
```



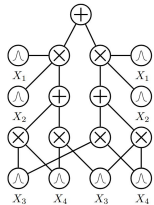
a notebook on learning SOS subtractive mixtures

`https://github.com/april-tools/cirkit/blob/main/notebooks/sum-of-squares-circuits.ipynb`

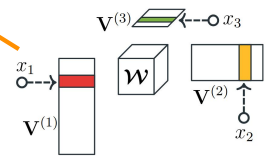
towards conclusions...



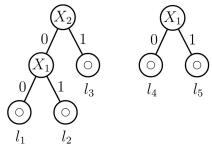
(hierarchical)
mixtures



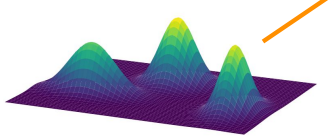
(deep)
circuits



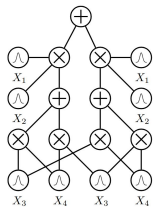
(hierarchical)
tensor factorizations



decision trees



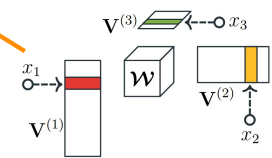
(hierarchical) mixtures



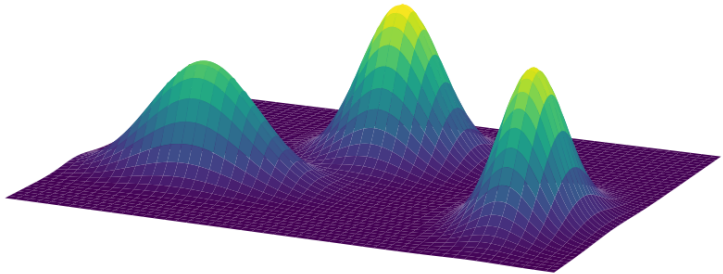
(deep) circuits

$$(d \to b) \wedge (e \to b)$$
$$\vdash (\neg d \vee b) \wedge (\neg e \vee b)$$
$$\vdash b \vee (\neg d \wedge \neg e)$$

logical formulas & constraints

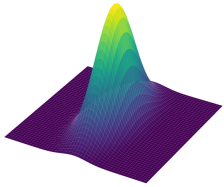


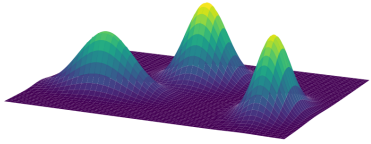
(hierarchical) tensor factorizations



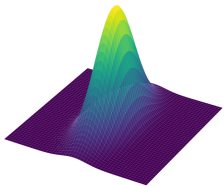
oh mixtures, you're so fine you blow my mind!

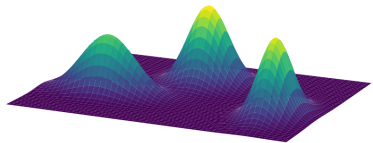
image taken from Hao Tang's course on ASR





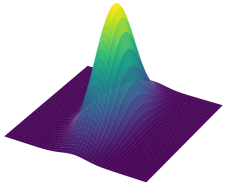
$$p(\mathbf{X}) \longrightarrow \sum_{i=1}^{K} w_i p_i(\mathbf{X}) \quad w_i > 0$$

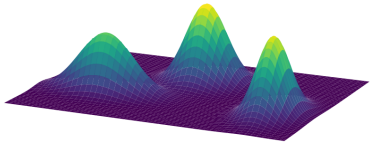


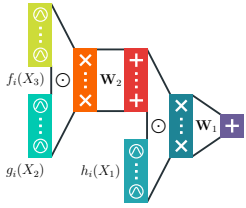


$$p(\mathbf{X}) \longrightarrow \sum_{i=1}^{K} w_i p_i(\mathbf{X}) \quad w_i > 0$$

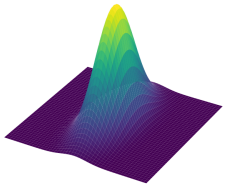
*"if someone publishes a paper on **model A**, there will be a paper about **mixtures of A** soon, with high probability"* **A. Vergari**

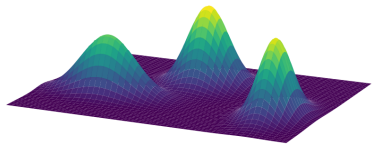


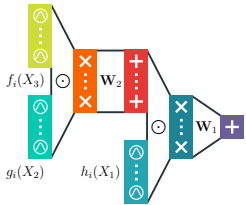




$$p(\mathbf{X}) \rightarrow \sum_{i=1}^{K} w_i p_i(\mathbf{X}) \quad w_i > 0 \rightarrow \sum_{i=1}^{2^D} w_i p_i(\mathbf{X}) = \mathsf{PC}(\mathbf{X})$$



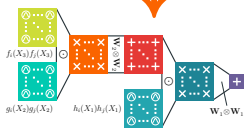




$$p(\mathbf{X}) \rightarrow \sum_{i=1}^{K} w_i p_i(\mathbf{X}) \quad w_i > 0 \rightarrow \sum_{i=1}^{2^D} w_i p_i(\mathbf{X}) = \mathsf{PC}(\mathbf{X})$$

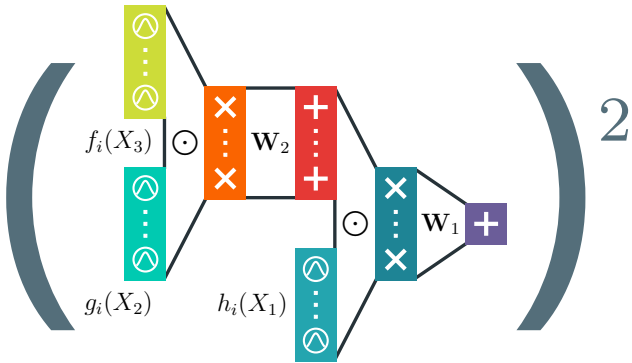
$$\downarrow \qquad\qquad\qquad\qquad\qquad\qquad \downarrow$$

$$\sum_j \left(\sum_{i=1}^{K} w_{ij} p_{ij}(\mathbf{X}) \right)^2 \quad w_{ij} \in \mathbb{R} \rightarrow$$



learning & reasoning with circuits in pytorch

`github.com/april-tools/cirkit`



questions?